```
Array.superclass        # => Object
Object.superclass       # => BasicObject
BasicObject.superclass  # => nil
```

The Array class inherits from Object, which is the same as saying "an array is an object." Object contains methods that are generally useful for any object—such as to_s, which converts an object to a string. In turn, Object inherits from BasicObject, the root of the Ruby class hierarchy, which contains only a few essential methods. (You will learn more about BasicObject later in the book.)

While talking about superclasses, we can ask ourselves one more question: what is the superclass of Class?

Modules

Take a deep breath and check out the superclass of the Class class itself:

```
Class.superclass        # => Module
```

The superclass of Class is Module—which is to say, every class is also a module. To be precise, a class is a module with three additional instance methods (new, allocate, and superclass) that allow you to create objects or arrange classes into hierarchies.

Indeed, classes and modules are so closely related that Ruby could easily get away with a single "thing" that plays both roles. The main reason for having a distinction between modules and classes is clarity: by carefully picking either a class or a module, you can make your code more explicit. Usually, you pick a module when you mean it to be included somewhere, and you pick a class when you mean it to be instantiated or inherited. So, although you can use classes and modules interchangeably in many situations, you'll probably want to make your intentions clear by using them for different purposes.

Putting It All Together

Bill concludes his lecture with a piece of code and a whiteboard diagram:

```
class MyClass; end
obj1 = MyClass.new
obj2 = MyClass.new
```